

```

98     String title = readText(parser);
99     title = title.replace("(v)". ""); // Remove (v) from end of title
100    parser.require(XmlPullParser.END_TAG, ns, "title");
101    return title;
102 }
103
104 // Processes pubDate tags in the feed.
105 private Date readPubDate(XmlPullParser parser) throws IOException, XmlPullParserException {
106     parser.require(XmlPullParser.START_TAG, ns, "pubDate");
107     String pubDate = readText(parser);
108     parser.require(XmlPullParser.END_TAG, ns, "pubDate");
109
110     //format date           Mon, 11 Mar 2013 01:00:00 +0000
111     SimpleDateFormat sdf = new SimpleDateFormat("EEE, dd MMM yyyy HH:mm:ss Z");//"yyyy-MM-dd'T'hh:
mm:ss'Z'");
112     Date sermonDate = new GregorianCalendar(0,0,0).getTime();
113     try {
114         sermonDate = sdf.parse(pubDate);
115     } catch (ParseException e) {
116         Log.d(TAG, "Date parsing exception.", e);
117     }
118     return sermonDate;
119 }
120
121 // Processes enclosure tags in the feed.
122 private String readUrl(XmlPullParser parser) throws IOException, XmlPullParserException {
123     String enclosure = "";
124     parser.require(XmlPullParser.START_TAG, ns, "enclosure");
125     String tag = parser.getName();
126     String url = parser.getAttributeValue(null, "url");
127     parser.nextTag();
128     parser.require(XmlPullParser.END_TAG, ns, "enclosure");
129     return url;
130 }
131
132 // Processes enclosure tags in the feed.
133 private String readThumbnailUrl(XmlPullParser parser) throws IOException, XmlPullParserException {
134     String enclosure = "";
135     parser.require(XmlPullParser.START_TAG, ns, "media:thumbnail");
136     String tag = parser.getName();
137     String url = parser.getAttributeValue(null, "url");
138     parser.nextTag();
139     parser.require(XmlPullParser.END_TAG, ns, "media:thumbnail");
140     return url;
141 }
142
143 // Processes summary tags in the feed.
144 private String readDescription(XmlPullParser parser) throws IOException, XmlPullParserException {
145     parser.require(XmlPullParser.START_TAG, ns, "description");

```

```
146     String summary = readText(parser);
147     parser.require(XmlPullParser.END_TAG, ns, "description");
148     return summary;
149 }
150
151 // For the tags title and summary, extracts their text values.
152 private String readText(XmlPullParser parser) throws IOException, XmlPullParserException {
153     String result = "";
154     if (parser.next() == XmlPullParser.TEXT) {
155         result = parser.getText();
156         parser.nextTag();
157     }
158     return result;
159 }
160
161 // Skips tags the parser isn't interested in. Uses depth to handle nested tags. i.e.,
162 // if the next tag after a START_TAG isn't a matching END_TAG, it keeps going until it
163 // finds the matching END_TAG (as indicated by the value of "depth" being 0).
164 private void skip(XmlPullParser parser) throws XmlPullParserException, IOException {
165     if (parser.getEventType() != XmlPullParser.START_TAG) {
166         throw new IllegalStateException();
167     }
168     int depth = 1;
169     while (depth != 0) {
170         switch (parser.next()) {
171             case XmlPullParser.END_TAG:
172                 depth--;
173                 break;
174             case XmlPullParser.START_TAG:
175                 depth++;
176                 break;
177         }
178     }
179 }
180 }
```

```

1 //package com.chris.churchapp.database;
2 //
3 //import android.content.Context;
4 //import android.database.sqlite.SQLiteDatabase;
5 //import android.database.sqlite.SQLiteOpenHelper;
6 //import android.util.Log;
7 //
8 //public class DbHelper extends SQLiteOpenHelper {
9 //
10 //    static final String TAG = "DbHelper";
11 //    private static final String DB_CREATE =
12 //        "create table " + SermonsContract.TABLE + "("
13 //        + SermonsContract.Columns.ID + " integer primary key autoincrement, "
14 //        + SermonsContract.Columns.TITLE + " text not null, "
15 //        + SermonsContract.Columns.PUBDATE + " INTEGER, "
16 //        + SermonsContract.Columns.DESCRPTION + " text, "
17 //        + SermonsContract.Columns.WATCHED_STATE + " INTEGER, "
18 //        + SermonsContract.Columns.URL + " text not null, "
19 //        + SermonsContract.Columns.THUMBNAIL_URL + " text);";
20 //
21 //    // Constructor
22 //    public DbHelper(Context context) {
23 //        super(context, SermonsContract.DB_NAME, null, SermonsContract.DB_VERSION);
24 //    }
25 //
26 //    // Called only once, first time the DB is created
27 //    @Override
28 //    public void onCreate(SQLiteDatabase db) {
29 //
30 //        Log.d(TAG, "onCreated sql: " + DB_CREATE);
31 //
32 //        db.execSQL(DB_CREATE);
33 //    }
34 //
35 //    // Called whenever newVersion != oldVersion
36 //    @Override
37 //    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
38 //        // Typically do ALTER TABLE statements, but...we're just in development,
39 //        // so:
40 //
41 //        db.execSQL("drop table if exists " + SermonsContract.TABLE); // remove the old database
42 //        // away
43 //        Log.d(TAG, "onUpdated");
44 //        onCreate(db); // run onCreate to get new database
45 //    }
46 //
47 //}
48
49

```

```

1 package com.chris.churchapp.database;
2
3 import android.content.ContentProvider;
4 import android.content.ContentUris;
5 import android.content.ContentValues;
6 import android.content.Context;
7 import android.content.UriMatcher;
8 import android.content.res.Resources;
9 import android.database.Cursor;
10 import android.database.SQLException;
11 import android.database.sqlite.SQLiteDatabase;
12 import android.database.sqlite.SQLiteOpenHelper;
13 import android.database.sqlite.SQLiteQueryBuilder;
14 import android.net.Uri;
15 import android.provider.LiveFolders;
16 import android.text.TextUtils;
17 import android.util.Log;
18
19 //import com.chris.churchapp.database.DbHelper;
20 import com.chris.churchapp.database.SermonsContract;
21
22 import java.util.HashMap;
23
24 public class SermonProvider extends ContentProvider {
25     public static String TAG = "SermonProvider";
26     public static final String DB_NAME = "sermons.db";
27     public static final int DB_VERSION = 1;
28
29     /**
30      * A projection map used to select columns from the database
31      */
32     private static HashMap<String, String> sSermonsProjectionMap;
33
34     /**
35      * Standard projection for the interesting columns of a normal note.
36      */
37     private static final String[] READ_SERMON_PROJECTION = new String[] {
38         SermonsContract.Columns.ID, // Projection position 0, the note's id
39         SermonsContract.Columns.DESCRPTION, // Projection position 1, the note's content
40         SermonsContract.Columns.TITLE, // Projection position 2, the note's title
41     };
42     private static final int READ_SERMON_DESCRIPTION_INDEX = 1;
43     private static final int READ_SERMON_TITLE_INDEX = 2;
44
45     /**
46      * Constants used by the Uri matcher to choose an action based on the pattern
47      * of the incoming URI
48      */
49     // The incoming URI matches the Sermons URI pattern

```

```

50 private static final int SERMONS = 1;
51
52 // The incoming URI matches the Sermon ID URI pattern
53 private static final int SERMON_ID = 2;
54
55 /**
56  * A UriMatcher instance
57  */
58 private static final UriMatcher sUriMatcher;
59
60 // Handle to a new DatabaseHelper.
61 private DatabaseHelper mOpenHelper;
62
63 /**
64  * A block that instantiates and sets static objects
65  */
66 static {
67
68     /*
69     * Creates and initializes the URI matcher
70     */
71     // Create a new instance
72     sUriMatcher = new UriMatcher(UriMatcher.NO_MATCH);
73
74     // Add a pattern that routes URIs terminated with "notes" to a NOTES operation
75     sUriMatcher.addURI(SermonsContract.AUTHORITY, "sermons", SERMONS); //ALLROWS);
76
77     // Add a pattern that routes URIs terminated with "notes" plus an integer
78     // to a note ID operation
79     sUriMatcher.addURI(SermonsContract.AUTHORITY, "sermons/#", SERMON_ID);
80
81     /*
82     * Creates and initializes a projection map that returns all columns
83     */
84
85     // Creates a new projection map instance. The map returns a column name
86     // given a string. The two are usually equal.
87     sSermonsProjectionMap = new HashMap<String, String>();
88
89     // Maps the string "_ID" to the column name "_ID"
90     sSermonsProjectionMap.put(SermonsContract.Columns._ID, SermonsContract.Columns._ID);
91
92     // Maps "title" to "title"
93     sSermonsProjectionMap.put(SermonsContract.Columns.TITLE, SermonsContract.Columns.TITLE);
94
95     // Maps "pubdate" to "pubdate"
96     sSermonsProjectionMap.put(SermonsContract.Columns.PUBDATE, SermonsContract.Columns.PUBDATE);
97
98     // Maps "note" to "Description"

```

```

99     sSermonsProjectionMap.put(SermonsContract.Columns.DESCRPTION, SermonsContract.Columns.DESCRIP
100     TION);
101     // Maps "watchedState" to "watchedState"
102     sSermonsProjectionMap.put(SermonsContract.Columns.WATCHED_STATE, SermonsContract.Columns.WAT
103     CHED_STATE);
104     // Maps "URL" to "URL"
105     sSermonsProjectionMap.put(SermonsContract.Columns.URL, SermonsContract.Columns.URL);
106
107     // Maps "thumbnailURL" to "thumbnailURL"
108     sSermonsProjectionMap.put(SermonsContract.Columns.THUMBNAI_URL, SermonsContract.Columns.THUM
109     BNAI_URL);
110 }
111
112 /**
113  *
114  * This class helps open, create, and upgrade the database file. Set to package visibility
115  * for testing purposes.
116  */
117 static class DatabaseHelper extends SQLiteOpenHelper {
118
119     static final String TAG = "DbHelper";
120
121     private static final String DB_CREATE =
122         "create table " + SermonsContract.TABLE + " ("
123             + SermonsContract.Columns.ID + " integer primary key autoincrement, "
124             + SermonsContract.Columns.TITLE + " text not null, "
125             + SermonsContract.Columns.PUBDATE + " INTEGER, "
126             + SermonsContract.Columns.DESCRPTION + " text, "
127             + SermonsContract.Columns.WATCHED_STATE + " INTEGER, "
128             + SermonsContract.Columns.URL + " text not null, "
129             + SermonsContract.Columns.THUMBNAI_URL + " text);";
130
131     // Constructor
132     DatabaseHelper(Context context) {
133         // calls the super constructor, requesting the default cursor factory.
134         super(context, DB_NAME, null, DB_VERSION);
135     }
136
137     // Called only once, first time the DB is created
138     @Override
139     public void onCreate(SQLiteDatabase db) {
140
141         Log.d(TAG, "onCreate sql: " + DB_CREATE);
142
143         /**
144          *

```

```

145     * Creates the underlying database with table name and column names taken from the
146     * NotePad class.
147     */
148     db.execSQL(DB_CREATE);
149 }
150
151 // Called whenever newVersion != oldVersion
152 @Override
153 public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
154
155     // Logs that the database is being upgraded
156     Log.w(TAG, "Upgrading database from version " + oldVersion + " to "
157           + newVersion + ", which will destroy all old data");
158
159     // Kills the table and existing data
160     db.execSQL("drop table if exists " + SermonsContract.TABLE);
161
162     // Recreates the database with a new version
163     onCreate(db);
164 }
165
166 }
167
168 /**
169  *
170  * Initializes the provider by creating a new DatabaseHelper. onCreate() is called
171  * automatically when Android creates the provider in response to a resolver request from a
172  * client.
173  */
174 @Override
175 public boolean onCreate() {
176
177     // Creates a new helper object. Note that the database itself isn't opened until
178     // something tries to access it, and it's only created if it doesn't already exist.
179     mOpenHelper = new DatabaseHelper(getContext());
180
181     // Assumes that any failures will be reported by a thrown exception.
182     return true;
183 }
184
185 /**
186  * This method is called when a client calls
187  * {@link android.content.ContentResolver#query(Uri, String[], String, String[], String)}.
188  * Queries the database and returns a cursor containing the results.
189  *
190  * @return A cursor containing the results of the query. The cursor exists but is empty if
191  * the query returns no results or an exception occurs.
192  * @throws IllegalArgumentException if the incoming URI pattern is invalid.
193  */

```

```

194 @Override
195 public Cursor query(Uri uri, String[] projection, String selection,
196     String[] selectionArgs, String sortOrder) {
197     // Log.v(TAG, "query: " + uri);
198     // Constructs a new query builder and sets its table name
199     SQLiteQueryBuilder qb = new SQLiteQueryBuilder();
200     qb.setTables(SermonsContract.TABLE);
201
202     /**
203      * Choose the projection and adjust the "where" clause based on URI pattern-matching.
204      */
205     switch (sUriMatcher.match(uri)) {
206         // If the incoming URI is for notes, chooses the Notes projection
207         case SERMONS:
208             qb.setProjectionMap(sSermonsProjectionMap);
209             break;
210
211         // If the incoming URI is for a single note identified by its ID, chooses the
212         // note ID projection, and appends "_ID = <noteID>" to the where clause, so that
213         // it selects that single note
214         case SERMON_ID:
215             qb.setProjectionMap(sSermonsProjectionMap);
216             qb.appendWhere(
217                 SermonsContract.Columns._ID + // the name of the ID column
218                 "=" +
219                 // the position of the sermon ID itself in the incoming URI
220                 uri.getPathSegments().get(
221                     SermonsContract.Columns.SERMON_ID_PATH_POSITION));
222             break;
223
224         default:
225             // If the URI doesn't match any of the known patterns, throw an exception.
226             throw new IllegalArgumentException("Unknown URI " + uri);
227     }
228
229     String orderBy;
230     // If no sort order is specified, uses the default
231     if (TextUtils.isEmpty(sortOrder)) {
232         orderBy = SermonsContract.SORT_ORDER;
233     } else {
234         // otherwise, uses the incoming sort order
235         orderBy = sortOrder;
236     }
237
238     // Opens the database object in "read" mode, since no writes need to be done.
239     SQLiteDatabase db = mOpenHelper.getReadableDatabase();
240
241     /**
242

```

```

243  * Performs the query. If no problems occur trying to read the database, then a Cursor
244  * object is returned; otherwise, the cursor variable contains null. If no records were
245  * selected, then the Cursor object is empty, and Cursor.getCount() returns 0.
246  */
247  Cursor c = qb.query(
248      db,          // The database to query
249      projection,  // The columns to return from the query
250      selection,   // The columns for the where clause
251      selectionArgs, // The values for the where clause
252      null,        // don't group the rows
253      null,        // don't filter by row groups
254      orderBy      // The sort order
255  );
256
257  // Tells the Cursor what URI to watch, so it knows when its source data changes
258  c.setNotificationUri(getContext().getContentResolver(), SermonsContract.Columns.CONTENT_URI); //uri);
259  return c;
260
261 }
262
263 /**
264  * This is called when a client calls {@link android.content.ContentResolver#getType(Uri)}.
265  * Returns the MIME data type of the URI given as a parameter.
266  *
267  * @param uri The URI whose MIME type is desired.
268  * @return The MIME type of the URI.
269  * @throws IllegalArgumentException if the incoming URI pattern is invalid.
270  */
271 @Override
272 public String getType(Uri uri) {
273     // TODO: Implement this to handle requests for the MIME type of the data
274     // at the given URI.
275     throw new UnsupportedOperationException("Not yet implemented");
276 }
277
278 /**
279  * This is called when a client calls
280  * {@link android.content.ContentResolver#insert(Uri, ContentValues)}.
281  * Inserts a new row into the database. This method sets up default values for any
282  * columns that are not included in the incoming map.
283  * If rows were inserted, then listeners are notified of the change.
284  * @return The row ID of the inserted row.
285  * @throws android.database.SQLException if the insertion fails.
286  */
287 @Override
288 public Uri insert(Uri uri, ContentValues initialValues) {
289     // Log.v(TAG, "insert: " + uri);
290     // Validates the incoming URI. Only the full provider URI is allowed for inserts.
291     if (sUriMatcher.match(uri) != SERMONS) {

```

```

292     throw new IllegalArgumentException("Unknown URI " + uri);
293 }
294
295 // A map to hold the new record's values.
296 ContentValues values;
297
298 // If the incoming values map is not null, uses it for the new values.
299 if (initialValues != null) {
300     values = new ContentValues(initialValues);
301 }
302 } else {
303     // Otherwise, create a new value map
304     values = new ContentValues();
305 }
306
307 // If the values map doesn't contain note text, sets the value to an empty string.
308 if (values.containsKey(SermonsContract.Columns.DESCRPTION) == false) {
309     values.put(SermonsContract.Columns.DESCRPTION, "");
310 }
311
312
313 // Opens the database object in "write" mode.
314 SQLiteDatabase db = mOpenHelper.getWritableDatabase();
315
316 // Performs the insert and returns the ID of the new note.
317 long rowId = db.insert(
318     SermonsContract.TABLE,           // The table to insert into.
319     SermonsContract.Columns.DESCRPTION, // A hack, SQLite sets this column value to null
320     // if values is empty.
321     values,                          // A map of column names, and the values to insert
322     // into the columns.
323 );
324
325 // If the insert succeeded, the row ID exists.
326 if (rowId > 0) {
327     // Creates a URI with the note ID pattern and the new row ID appended to it.
328     Uri sermonUri = ContentUris.withAppendedId(SermonsContract.Columns.CONTENT_ID_URI_BASE, rowId);
329
330     // Notifies observers registered against this provider that the data changed.
331     getContext().getContentResolver().notifyChange(sermonUri, null);
332     return sermonUri;
333 }
334
335 // If the insert didn't succeed, then the rowId is <= 0. Throws an exception.
336 throw new SQLException("Failed to insert row into " + uri);
337
338 }
339
340 /**

```

```

341  * This is called when a client calls
342  * {@link android.content.ContentResolver#update(Uri, ContentValues, String, String[])}
343  * Updates records in the database. The column names specified by the keys in the values map
344  * are updated with new data specified by the values in the map. If the incoming URI matches the
345  * note ID URI pattern, then the method updates the one record specified by the ID in the URI;
346  * otherwise, it updates a set of records. The record or records must match the input
347  * selection criteria specified by where and whereArgs.
348  * If rows were updated, then listeners are notified of the change.
349  *
350  * @param uri The URI pattern to match and update.
351  * @param values A map of column names (keys) and new values (values).
352  * @param where An SQL "WHERE" clause that selects records based on their column values. If this
353  * is null, then all records that match the URI pattern are selected.
354  * @param whereArgs An array of selection criteria. If the "where" param contains value
355  * placeholders ("?"), then each placeholder is replaced by the corresponding element in the
356  * array.
357  * @return The number of rows updated.
358  * @throws IllegalArgumentException if the incoming URI pattern is invalid.
359  */
360  @Override
361  public int update(Uri uri, ContentValues values, String where, String[] whereArgs) {
362
363      // Opens the database object in "write" mode.
364      SQLiteDatabase db = mOpenHelper.getWritableDatabase();
365      int count;
366      String finalWhere;
367
368      // Does the update based on the incoming URI pattern
369      switch (sUriMatcher.match(uri)) {
370          case SERMONS:
371
372              // Does the update and returns the number of rows updated.
373              count = db.update(
374                  SermonsContract.TABLE, // The database table name.
375                  values, // A map of column names and new values to use.
376                  where, // The where clause column names.
377                  whereArgs // The where clause column values to select on.
378              );
379              break;
380
381              // If the incoming URI matches a single note ID, does the update based on the incoming
382              // data, but modifies the where clause to restrict it to the particular note ID.
383          case SERMON_ID:
384              // From the incoming URI, get the note ID
385              String sermonId = uri.getPathSegments().get(SermonsContract.Columns.SERMON_ID_PATH_POSITIO
N);
386
387              /*
388              * Starts creating the final WHERE clause by restricting it to the incoming

```

```

389      * note ID.
390      */
391      finalWhere =
392          SermonsContract.Columns._ID +
393          " = " +
394          uri.getPathSegments().get(SermonsContract.Columns.SERMON_ID_PATH_POSITION);
395
396      // If there were additional selection criteria, append them to the final WHERE
397      // clause
398      if (where != null) {
399          finalWhere = finalWhere + " AND " + where;
400      }
401
402      // Does the update and returns the number of rows updated.
403      count = db.update(
404          SermonsContract.TABLE, // The database table name.
405          values, // A map of column names and new values to use.
406          finalWhere, // The final WHERE clause to use
407          // placeholders for whereArgs
408          whereArgs // The where clause column values to select on, or
409                  // null if the values are in the where argument.
410      );
411      break;
412      // If the incoming pattern is invalid, throws an exception
413      default: throw new IllegalArgumentException("Unknown URI " + uri);
414  }
415
416  /*Gets a handle to the content resolver object for the current context, and notifies it
417  * that the incoming URI changed. The object passes this along to the resolver framework,
418  * and observers that have registered themselves for the provider are notified.
419  */
420  getContext().getContentResolver().notifyChange(uri, null);
421
422  // Returns the number of rows updated.
423  return count;
424  }
425
426  @Override
427  public int delete(Uri uri, String selection, String[] selectionArgs) {
428      // Implement this to handle requests to delete one or more rows.
429      throw new UnsupportedOperationException("Not yet implemented");
430  }
431  }
432

```

```

1 package com.chris.churchapp.database;
2
3 import android.net.Uri;
4 import android.provider.BaseColumns;
5
6 /**
7  * Defines a contract between the sermon content provider and its clients. A contract defines the
8  * information that a client needs to access the provider as one or more data tables. A contract
9  * is a public, non-extendable (final) class that contains constants defining column names and
10  * URIs. A well-written client depends only on the constants in the contract.
11  */
12 public final class SermonsContract {
13     public static final String AUTHORITY = "com.chris.churchapp.database.SermonProvider";
14
15     public static final String TABLE = "sermons";
16     public static final String SORT_ORDER = Columns.PUBDATE + " desc";
17
18     //This class cannot be instantiated
19     private SermonsContract(){}
20
21     public static final class Columns implements BaseColumns {
22         private Columns(){}
23
24         /**
25          * 0-relative position of a sermon ID segment in the path part of a sermon ID URI
26          */
27         public static final int SERMON_ID_PATH_POSITION = 1;
28
29         /**
30          * The scheme part for this provider's URI
31          */
32         private static final String SCHEME = "content://";
33
34         /**
35          * Path part for the Sermons URI
36          */
37         private static final String PATH_SERMONS = "/sermons";
38
39         /**
40          * Path part for the sermon ID URI
41          */
42         private static final String PATH_SERMON_ID = "/sermons/";
43
44         /**
45          * The content:// style URL for this table
46          */
47         public static final Uri CONTENT_URI = Uri.parse(SCHEME + AUTHORITY + PATH_SERMONS);
48
49         /**
50          * The content URI base for a single sermon. Callers must

```

```

50      * append a numeric sermon id to this Uri to retrieve a sermon
51      */
52      public static final Uri CONTENT_ID_URI_BASE
53          = Uri.parse(SCHEME + AUTHORITY + PATH_SERMON_ID);
54
55      /**
56       * The content URI match pattern for a single sermon, specified by its ID. Use this to match
57       * incoming URIs or to construct an Intent.
58       */
59      public static final Uri CONTENT_ID_URI_PATTERN
60          = Uri.parse(SCHEME + AUTHORITY + PATH_SERMON_ID + "/" + "#");
61
62      /*
63       * Column definitions
64       */
65
66      /**
67       * Column name for the title of the sermon
68       * <P>Type: TEXT</P>
69       */
70      public static final String COLUMN_NAME_TITLE = "title";
71
72      /**
73       * Column name of the sermon content
74       * <P>Type: TEXT</P>
75       */
76      public static final String COLUMN_NAME_DESCRIPTION = "sermon";
77
78      public static final String TITLE = "sermon_title";
79      public static final String PUBDATE = "sermon_pubdate";
80      public static final String DESCRIPTION = "sermon_description";
81      public static final String URL = "sermon_video_url";
82      public static final String THUMBNAIL_URL = "sermon_video_thumbnail_url";
83      public static final String WATCHED_STATE = "watched_state";
84  }
85
86 }
87

```

```
1 package com.chris.churchapp.database;
2
3 import android.content.SearchRecentSuggestionsProvider;
4
5 public class SermonSuggestionProvider extends SearchRecentSuggestionsProvider {
6     public final static String AUTHORITY = "com.chris.churchapp.database.SermonSuggestionProvider";
7     public final static int MODE = DATABASE_MODE_QUERIES;
8
9     public SermonSuggestionProvider() {
10         setupSuggestions(AUTHORITY, MODE);
11     }
12 }
13
```

```

1 package com.chris.churchapp.fragments;
2
3 import android.app.Activity;
4 import android.app.ProgressDialog;
5 import android.database.Cursor;
6 import android.graphics.Typeface;
7 import android.os.Bundle;
8 import android.app.ListFragment;
9 import android.app.LoaderManager;
10 import android.content.Loader;
11 import android.content.CursorLoader;
12 import android.text.format.DateUtils;
13 import android.util.Log;
14 import android.view.Gravity;
15 import android.view.LayoutInflater;
16 import android.view.Menu;
17 import android.view.MenuInflater;
18 import android.view.View;
19 import android.view.ViewGroup;
20 import android.widget.FrameLayout;
21 import android.widget.ImageView;
22 import android.widget.ListView;
23 import android.widget.ProgressBar;
24 import android.widget.SearchView;
25 import android.widget.SimpleCursorAdapter;
26 import android.widget.SimpleCursorAdapter.ViewBinder;
27 import android.widget.TextView;
28 import com.chris.churchapp.R;
29 import com.chris.churchapp.SermonItem;
30 import com.chris.churchapp.database.SermonsContract;
31 import com.nostr13.universalimageloader.core.DisplayImageOptions;
32 import com.nostr13.universalimageloader.core.ImageLoader;
33 import java.util.ArrayList;
34
35 public class SermonListFragment extends ListFragment implements
36 LoaderManager.LoaderCallbacks<Cursor> {
37
38     private static final String TAG = "SermonListFragment";
39
40     /**
41      * The serialization (saved instance state) Bundle key representing the
42      * activated item position. Only used on tablets.
43      */
44     private static final String STATE_ACTIVATED_POSITION = "activated_position";
45
46     /**
47      * The current activated item position. Only used on tablets.
48      */
49     private int mActivatedPosition = ListView.INVALID_POSITION;

```

```

50
51 private static final String[] FROM = {SermonsContract.Columns.TITLE,
52     SermonsContract.Columns.PUBDATE, SermonsContract.Columns.THUMBNAI_URL};
53 private static final int TO[] = {R.id.sermon_title_textview,
54     R.id.sermon_date_textview, R.id.sermon_thumbnail_imageview};
55
56 private ProgressDialog pDialog;
57
58 SimpleCursorAdapter simpleCursorAdapter;
59
60 OnSermonSelectedListener mSermonSelectedListener;
61
62 /**
63  * Mandatory empty constructor for the fragment manager to instantiate the
64  * fragment (e.g. upon screen orientation changes).
65  */
66 public SermonListFragment() {}
67
68 // The container Activity must implement this interface so the frag can deliver messages
69 public interface OnSermonSelectedListener {
70     /** Called by SermonListFragment when a list item is selected */
71     public void onSermonSelected(Long id);
72 }
73
74 /**
75  * Sets the listener that should be notified of headline selection events.
76  * @param listener the listener to notify.
77  */
78 public void setOnSermonSelectedListener(OnSermonSelectedListener listener) {
79     mSermonSelectedListener = listener;
80 }
81
82 @Override
83 public void onCreate(Bundle savedInstanceState) {
84     super.onCreate(savedInstanceState);
85     Log.i(TAG, "onCreate");
86
87     simpleCursorAdapter = new SimpleCursorAdapter(
88         getActivity(),
89         R.layout.sermon_list_item,
90         null,
91         FROM,
92         TO,
93         0
94     );
95
96     simpleCursorAdapter.setViewBinder(new SermonViewBinder());
97     setListAdapter(simpleCursorAdapter);
98

```

```

99  }
100
101  @Override
102  public void onViewCreated(View view, Bundle savedInstanceState) {
103      super.onViewCreated(view, savedInstanceState);
104
105      // Restore the previously serialized activated item position.
106      if (savedInstanceState != null
107          && savedInstanceState.containsKey(STATE_ACTIVATED_POSITION)) {
108          setActivatedPosition(savedInstanceState.getInt(STATE_ACTIVATED_POSITION));
109      }
110  }
111
112
113  @Override
114  public void onCreateOptionsMenu(Menu menu, MenuInflater inflater) {
115      super.onCreateOptionsMenu(menu, inflater);
116      menu.findItem(R.id.menu_search).getActionView();
117  }
118  }
119
120
121  @Override
122  public void onActivityCreated(Bundle savedInstanceState) {
123      super.onActivityCreated(savedInstanceState);
124      Log.i(TAG, "onActivityCreated");
125
126      setEmptyText("No sermons found");
127      getLoaderManager().initLoader(0, null, this);
128  }
129  }
130
131  @Override
132  public void onAttach(Activity activity) {
133      super.onAttach(activity);
134      Log.i(TAG, "onAttach");
135
136      // This makes sure that the container activity has implemented
137      // the callback interface. If not, it throws an exception.
138      try {
139          mSermonSelectedListener = (OnSermonSelectedListener) activity;
140      } catch (ClassCastException e) {
141          throw new ClassCastException(activity.toString()
142              + " must implement OnSermonSelectedListener");
143      }
144  }
145
146  @Override
147  public void onStart() {

```

```

148     super.onStart();
149     Log.i(TAG, "onStart");
150
151     // When in two-pane layout, set the listview to highlight the selected list item
152     // (We do this during onStart because at the point the listview is available.)
153     if (getFragmentManager().findFragmentById(R.id.sermon_detail_container) != null) {
154         getListView().setChoiceMode(ListView.CHOICE_MODE_SINGLE);
155         setActivateOnItemClick(true);
156     }
157 }
158
159 /** Handles custom binding of data to view. */
160 class SermonViewBinder implements ViewBinder {
161
162     @Override
163     public boolean setViewValue(View view, Cursor cursor, int columnIndex) {
164         if (view.getId() == R.id.sermon_title_textview) {
165             if (cursor.getInt(
166                 cursor.getColumnIndex(
167                     SermonsContract.Columns.WATCHED_STATE)) == 0) {
168                 ((TextView) view).setTypeface(null, Typeface.BOLD);
169             }
170             else {
171                 ((TextView) view).setTypeface(null, Typeface.NORMAL);
172             }
173         }
174         if (view.getId() == R.id.sermon_date_textview) {
175             // Convert timestamp to relative time
176             Long timestamp = cursor.getLong(columnIndex);
177             CharSequence relativeTime = DateUtils
178                 .getRelativeTimeSpanString(timestamp);
179             ((TextView) view).setText(relativeTime);
180
181             return true;
182         }
183         else if (view.getId() == R.id.sermon_thumbnail_imageview) {
184             ImageLoader.getInstance().displayImage(cursor.getString(columnIndex),
185                 (ImageView) view);
186
187             return true;
188         } else return false;
189     }
190 }
191
192 @Override
193 public void onResume() {
194     super.onResume();
195     Log.i(TAG, "OnResume");
196 }

```

```

197
198 @Override
199 public void onListItemClick(ListView l, View v, int position, long id) {
200     super.onListItemClick(l, v, position, id);
201     Log.i(TAG, "onListItemClick " + Integer.toString(position));
202
203     mSermonSelectedListener.onSermonSelected(id);
204     setActivatedPosition(position);
205 }
206
207
208 /**
209  * Turns on activate-on-click mode. When this mode is on, list items will be
210  * given the 'activated' state when touched.
211  */
212 public void setActivateOnItemClick(boolean activateOnItemClick) {
213     // When setting CHOICE_MODE_SINGLE, ListView will automatically
214     // give items the 'activated' state when touched.
215     getListView().setChoiceMode(activateOnItemClick
216         ? ListView.CHOICE_MODE_SINGLE
217         : ListView.CHOICE_MODE_NONE);
218 }
219
220 private void setActivatedPosition(int position) {
221     if (position == ListView.INVALID_POSITION) {
222         getListView().setItemChecked(mActivatedPosition, false);
223     } else {
224         getListView().setItemChecked(position, true);
225     }
226
227     mActivatedPosition = position;
228 }
229
230 @Override
231 public Loader<Cursor> onCreateLoader(int id, Bundle args) {
232     Log.i(TAG, "onCreateLoader");
233
234     return new CursorLoader(getActivity(), SermonsContract.Columns.CONTENT_URI,
235         null, null, null, null);
236 }
237
238 @Override
239 public void onLoadFinished(Loader<Cursor> arg0, Cursor c) {
240     Log.i(TAG, "onLoadFinished");
241
242     simpleCursorAdapter.swapCursor(c);
243 }
244
245

```

```
246  @Override
247  public void onLoaderReset(Loader<Cursor> arg0) {
248      Log.i(TAG, "onLoaderReset");
249      simpleCursorAdapter.swapCursor(null);
250  }
251 }
```

```

1 package com.chris.churchapp.fragments;
2
3 import android.content.ContentResolver;
4 import android.content.ContentUris;
5 import android.content.ContentValues;
6 import android.content.Intent;
7 import android.database.Cursor;
8 import android.net.Uri;
9 import android.os.Bundle;
10 import android.text.Html;
11 import android.text.method.ScrollingMovementMethod;
12 import android.util.Log;
13 import android.view.LayoutInflater;
14 import android.view.View;
15 import android.view.View.OnClickListener;
16 import android.view.ViewGroup;
17 import android.widget.ImageButton;
18 import android.widget.ImageView;
19 import android.widget.TextView;
20 import android.app.Fragment;
21 import com.chris.churchapp.R;
22 import com.chris.churchapp.SermonItem;
23 import com.chris.churchapp.database.SermonsContract;
24 import com.nostr3l3.universalimageloader.core.ImageLoader;
25
26 public class SermonDetailFragment extends Fragment {
27
28     /**
29      * The fragment argument representing the item ID that this fragment
30      * represents.
31      */
32     public static final String ARG_ITEM_ID = "item_id";
33
34     private static final String TAG = "SermonDetailFragment";
35
36     int mCurrentPosition;
37     Long mCurrentID;
38     ImageView mImageView;
39     TextView sermonTitleTextView;
40     TextView sermonDetailTextView;
41     String videoUrl;
42     ImageButton mButton;
43
44     private SermonItem mItem;
45
46     /**
47      * Mandatory empty constructor for the fragment manager to instantiate the
48      * fragment (e.g. upon screen orientation changes).

```

```

50  */
51  public SermonDetailFragment() {
52      super();
53  }
54
55  @Override
56  public void onCreate(Bundle savedInstanceState){
57      super.onCreate(savedInstanceState);
58      Log.i(TAG, "onCreate");
59
60      mCurrentID = getArguments() != null ? getArguments().getLong(ARG_ITEM_ID) : 1;
61
62  }
63  @Override
64  public View onCreateView(LayoutInflater inflater, ViewGroup container,
65      Bundle savedInstanceState) {
66      super.onCreate(savedInstanceState);
67      Log.i(TAG, "onCreateView");
68
69      View rootView = inflater.inflate(R.layout.fragment_sermon_detail, container, false);
70
71      // setContentView(R.layout.activity_sermon_detail);
72
73      sermonTitleTextView = (TextView) rootView.findViewById(R.id.sermon_detail_title_textView);
74      mImageView = (ImageView) rootView.findViewById(R.id.sermon_detail_imageView);
75      sermonDetailTextView = (TextView) rootView.findViewById(R.id.sermon_detail_textView);
76      mButton = (ImageButton) rootView.findViewById(R.id.sermon_detail_button);
77
78      sermonDetailTextView.setMovementMethod(new ScrollingMovementMethod());
79
80      updateSermonView(mCurrentID);
81
82
83      // return inflater.inflate(R.id.sermon_detail_container, container, false);
84      return rootView;
85  }
86
87  @Override
88  public void onStart() {
89      super.onStart();
90
91      Log.i(TAG, "onStart");
92
93      // ImageButton mButton = (ImageButton) getActivity().findViewById(R.id.sermon_detail_button);
94      mButton.setOnClickListener(new OnClickListener() {
95          @Override
96          public void onClick(View v) {
97              if (imageUrl.isEmpty()) {
98                  Uri sermonUri = ContentUris.withAppendedId(SermonsContract.Columns.CONTENT_URI, mCurrentID

```

```

99         ContentResolver cr = getActivity().getContentResolver();
100
101         ContentValues values = new ContentValues();
102
103         values.put(SermonsContract.Columns.WATCHED_STATE, 1);
104
105         cr.update(sermonUri, values, null, null);
106
107
108         Uri uriUrl = Uri.parse(videoUrl);
109         Intent intent = new Intent(
110             android.content.Intent.ACTION_VIEW);
111         intent.setDataAndType(uriUrl, "video/mp4");
112         startActivity(intent);
113     }
114 }
115 });
116
117 // During startup, check if there are arguments passed to the fragment.
118 // onStart is a good place to do this because the layout has already been
119 // applied to the fragment at this point so we can safely call the method
120 // below that sets the article text.
121 Bundle args = getArguments();
122 if (args != null) {
123     // Set article based on argument passed in
124     updateSermonView(args.getLong(ARG_ITEM_ID));
125     Log.i(TAG, "Start up args: " + args.getLong(ARG_ITEM_ID));
126 }
127 }
128
129 public void updateSermonView(Long id) {
130
131     mCurrentId = id;
132
133     ContentResolver contentResolver = getActivity().getContentResolver();
134
135     // Construct a where clause to make sure we don't already have this
136     // sermon in the provider.
137     String w = SermonsContract.Columns._ID + " = " + id;
138
139     // Constructs a new URI from the incoming URI and the row ID
140     Uri uri = ContentUris.withAppendedId(SermonsContract.Columns.CONTENT_URI, id);
141
142     // If the earthquake is new, insert it into the provider.
143     Cursor result = contentResolver.query(uri, null, null, null, null);
144
145     // if (query.getCount()==0) {
146     if(result.moveToFirst()){
147         sermonTitleTextView.setText(result.getString(

```

```
148         result.getColumnIndex(SermonsContract.Columns.TITLE));
149         videoUrl =
150             result.getString(
151                 result.getColumnIndex(SermonsContract.Columns.URL));
152         String thumbnailUrlString =
153             result.getString(
154                 result.getColumnIndex(SermonsContract.Columns.THUMBNAI_URL));
155         // Start the DownloadImage task with the given url
156         ImageLoader.getInstance().displayImage(thumbnailUrlString, mImageView);
157         sermonDetailTextView.setText(Html.fromHtml(
158             result.getString(
159                 result.getColumnIndex(SermonsContract.Columns.DESCRPTION)
160             )
161         )
162         );
163     }
164 }
165
166 @Override
167 public void onSaveInstanceState(Bundle outState) {
168     super.onSaveInstanceState(outState);
169
170     // Save the current article selection in case we need to recreate the fragment
171     outState.putLong(ARG_ITEM_ID, mCurrentID);
172 }
173
174 }
175
```

```

1 package com.chris.churchapp.activities;
2
3 import android.app.Activity;
4 import android.content.Intent;
5 import android.database.Cursor;
6 import android.net.Uri;
7 import android.os.Bundle;
8 import android.app.LoaderManager;
9 import android.content.CursorLoader;
10 import android.content.Loader;
11 import android.util.Log;
12 import android.view.View;
13 import android.widget.AdapterView;
14 import android.widget.AdapterView.Adapter;
15 import android.widget.AdapterView.OnItemClickListener;
16 import android.widget.AdapterView.OnItemClickListener;
17 import android.widget.AdapterView.OnItemClickListener;
18 import android.widget.AdapterView.OnItemClickListener;
19 import android.widget.AdapterView.OnItemClickListener;
20 import android.widget.AdapterView.OnItemClickListener;
21 import android.widget.AdapterView.OnItemClickListener;
22 import android.widget.AdapterView.OnItemClickListener;
23 import android.widget.AdapterView.OnItemClickListener;
24
25
26 public class MainActivity extends Activity implements LoaderManager.LoaderCallbacks<Cursor> {
27
28     private static final String TAG = "MainActivity";
29
30     ImageView mImageView;
31     LinearLayout mLinearLayout;
32     Long newestSermonId;
33     ListView listView;
34     ProgressBar mProgressBar;
35     TextView sermonTitleTextView;
36
37     public void onCreate(Bundle savedInstanceState) {
38         super.onCreate(savedInstanceState);
39         setContentView(R.layout.activity_main);
40
41         mImageView = (ImageView) findViewById(R.id.imageView);
42         listView = (ListView) findViewById(R.id.main_listView);
43         mLinearLayout = (LinearLayout) findViewById(R.id.sermon_linear_layout);
44         mProgressBar = (ProgressBar) findViewById(R.id.progressBar);
45         sermonTitleTextView = (TextView) findViewById(R.id.main_sermon_title_textView);
46
47         String[] listItems = getResources().getStringArray(R.array.main_list_items);
48
49         //Binding resources to ListAdapter

```

```

50 listView.setAdapter(new ArrayAdapter<String>(this, android.R.layout.simple_list_item_1,
51     android.R.id.text1, listItems));
52
53 listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
54     public void onItemClick(AdapterView<?> adapterView, View view, int position, long id) {
55         Log.i(TAG, "listView click: " + position);
56         switch (position){
57             case 0: {
58                 Intent sermonListIntent = new Intent(getApplicationContext(), SermonListActivity.class);
59                 sermonListIntent.putExtra(SermonDetailFragment.ARG_ITEM_ID, newestSermonId);
60                 startActivity(sermonListIntent);
61                 break;
62             }
63             case 1: {
64                 String url = "http://journeychurchsc.org/";
65                 Intent i = new Intent(Intent.ACTION_VIEW);
66                 i.setData(Uri.parse(url));
67                 startActivity(i);
68                 break;
69             }
70         }
71     }
72 });
73
74 // updateSermonView(id);
75 getLoaderManager().initLoader(1, null, this);
76 }
77
78 @Override
79 public Loader<Cursor> onCreateLoader(int i, Bundle bundle) {
80     return new CursorLoader(this, SermonsContract.Columns.CONTENT_URI, null, null, null, null);
81 }
82
83 @Override
84 public void onLoadFinished(Loader<Cursor> cursorLoader, Cursor cursor) {
85     if(cursor.moveToFirst()){
86         newestSermonId = cursor.getLong(cursor.getColumnIndex(SermonsContract.Columns._ID));
87         sermonTitleTextView.setText(
88             cursor.getString(
89                 cursor.getColumnIndex(SermonsContract.Columns.TITLE));
90         sermonTitleTextView.setVisibility(View.VISIBLE);
91         String thumbnailUrlString =
92             cursor.getString(
93                 cursor.getColumnIndex(SermonsContract.Columns.THUMBNAI_URL));
94         ImageLoader.getInstance().displayImage(thumbnailUrlString, mImageView);
95         mImageView.setVisibility(View.VISIBLE);
96         mProgressBar.setVisibility(View.GONE);
97
98         mLinearLayout.setOnClickListener(new View.OnClickListener() {

```

```
99         @Override
100         public void onClick(View v) {
101
102             Intent detailIntent = new Intent(getApplicationContext(), SermonDetailActivity.class);
103             detailIntent.putExtra(SermonDetailFragment.ARG_ITEM_ID, newestSermonId);
104             startActivity(detailIntent);
105         }
106     });
107 }
108 }
109
110 @Override
111 public void onLoaderReset(Loader<Cursor> cursorLoader) {
112
113 }
114 }
```

```

1 package com.chris.churchapp.activities;
2
3 import android.app.ActionBar;
4 import android.app.SearchManager;
5 import android.content.Context;
6 import android.content.Intent;
7 import android.net.ConnectivityManager;
8 import android.net.NetworkInfo;
9 import android.os.Bundle;
10 import android.app.Activity;
11 import android.util.Log;
12 import android.view.Menu;
13 import android.view.MenuItem;
14 import android.view.View;
15 import android.widget.SearchView;
16 import android.widget.Toast;
17
18 import com.chris.churchapp.R;
19 import com.chris.churchapp.RefreshService;
20 import com.chris.churchapp.fragments.SermonDetailFragment;
21 import com.chris.churchapp.fragments.SermonListFragment;
22 import com.chris.churchapp.fragments.SermonListFragment.OnSermonSelectedListener;
23
24 public class SermonListActivity extends Activity implements //ActionBar.OnNavigationListener,
25     OnSermonSelectedListener{
26
27     private static final String TAG = "SermonListActivity";
28     private boolean mIsDualPane;
29
30     SermonListFragment mSermonListFragment;
31     SermonDetailFragment mSermonDetailFragment;
32
33     Long mSermonIndex = 1L;
34
35     /**
36      * The serialization (saved instance state) Bundle key representing the
37      * current dropdown position.
38      */
39     private static final String STATE_SELECTED_NAVIGATION_ITEM = "selected_navigation_item";
40
41     @Override
42     protected void onCreate(Bundle savedInstanceState) {
43         super.onCreate(savedInstanceState);
44         Log.i(TAG, "onCreate");
45         setContentView(R.layout.activity_sermon_list);
46
47         final ActionBar actionBar = getActionBar();
48         actionBar.setDisplayHomeAsUpEnabled(true);
49

```

```

50 // find our fragments
51 mSermonListFragment = (SermonListFragment) getFragmentManager().findFragmentById(
52     R.id.sermon_list);
53 mSermonDetailFragment = (SermonDetailFragment) getFragmentManager().findFragmentById(
54     R.id.sermon_detail_container);
55
56 mSermonListFragment.setOnSermonSelectedListener(this);
57
58 // Determine whether we are in single-pane or dual-pane mode by testing the visibility
59 // of the article view.
60 View SermonDetailsView = findViewById(R.id.sermon_detail_container);
61 mIsDualPane = SermonDetailsView != null && SermonDetailsView.getVisibility() == View.VISIBLE;
62
63 mSermonListFragment.setActivateOnItemClick(mIsDualPane);
64 restoreSelection(savedInstanceState);
65
66 }
67
68 @Override
69 public boolean onCreateOptionsMenu(Menu menu) {
70     getMenuInflater().inflate(R.menu.activity_main, menu);
71
72     // Get the SearchView and set the searchable configuration
73     SearchManager searchManager = (SearchManager) getSystemService(Context.SEARCH_SERVICE);
74     SearchView searchView = (SearchView) menu.findItem(R.id.menu_search).getActionView();
75     // Assumes current activity is the searchable activity
76     searchView.setSearchableInfo(searchManager.getSearchableInfo(getComponentName()));
77     searchView.setIconifiedByDefault(false); // Do not iconify the widget; expand it by default
78     return true;
79 }
80
81 @Override
82 public boolean onOptionsItemSelected(MenuItem item) {
83     super.onOptionsItemSelected(item);
84
85     switch (item.getItemId()) {
86         case android.R.id.home:
87             // This ID represents the Home or Up button. In the case of this
88             // activity, the Up button is shown. Use NavUtils to allow users
89             // to navigate up one level in the application structure. For
90             // more details, see the Navigation pattern on Android Design:
91             //
92             // http://developer.android.com/design/patterns/navigation.html#up-vs-back
93             //
94             navigateUpTo(this, new Intent(this, MainActivity.class));
95             return true;
96         case (R.id.menu_refresh): {
97
98             Context context = getApplicationContext();

```

```

99     int duration = Toast.LENGTH_SHORT;
100     CharSequence text;
101
102     ConnectivityManager connMgr = (ConnectivityManager)
103         getSystemService(Context.CONNECTIVITY_SERVICE);
104     NetworkInfo networkInfo = connMgr.getActiveNetworkInfo();
105     if (networkInfo != null && networkInfo.isConnected()) {
106         text = "Refreshing sermons";
107         startService(new Intent(this, RefreshService.class));
108     } else {
109         text = "No network connection available.";
110         Log.v(TAG, "No network connection available.");
111         // Toast.makeText(getApplicationContext(), "No network connection available.", Toast.LENGTH_SHORT
112         );
113     }
114     Toast toast = Toast.makeText(context, text, duration);
115     toast.show();
116     return true;
117 }
118 default: return false;
119 }
120
121 public void navigateUpTo(Activity activity, Intent upIntent) {
122     upIntent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);
123     activity.startActivity(upIntent);
124     activity.finish();
125 }
126
127 /** Save instance state. Saves current category/article index. */
128 @Override
129 protected void onSaveInstanceState(Bundle outState) {
130
131     outState.putLong(SermonDetailFragment.ARG_ITEM_ID, mSermonIndex);
132     super.onSaveInstanceState(outState);
133 }
134
135 /** Restore category/article selection from saved state. */
136 void restoreSelection(Bundle savedInstanceState) {
137     if (savedInstanceState != null) {
138         // setNewsCategory(savedInstanceState.getLong("catIndex", 0));
139         if (mIsDualPane) {
140             mSermonIndex = savedInstanceState.getLong(SermonDetailFragment.ARG_ITEM_ID, 1L);
141             //mSermonListFragment.setSelection(0); //Needs to be fixed to send pos
142             onSermonSelected(mSermonIndex);
143         }
144     }
145 }
146

```

```
147  @Override
148  public void onRestoreInstanceState(Bundle savedInstanceState) {
149      // Always call the superclass so it can restore the view hierarchy
150      super.onRestoreInstanceState(savedInstanceState);
151
152      restoreSelection(savedInstanceState);
153  }
154
155  @Override
156  public void onSermonSelected(Long id) {
157      // The user selected the title of a sermon from the SermonListFragment
158      mSermonIndex = id;
159      if(misDualPane) {
160          // In two-pane mode, show the detail view in this activity by
161          // adding or replacing the detail fragment using a
162          // fragment transaction.
163
164          mSermonDetailFragment.updateSermonView(mSermonIndex);
165
166      } else {
167          // In single-pane mode, simply start the detail activity
168          // for the selected item ID.
169          Intent detailIntent = new Intent(this, SermonDetailActivity.class);
170          detailIntent.putExtra(SermonDetailFragment.ARG_ITEM_ID, mSermonIndex);
171          startActivity(detailIntent);
172      }
173  }
174 }
```

```

1 package com.chris.churchapp.activities;
2
3 import android.app.Activity;
4 import android.content.Intent;
5 import android.os.Bundle;
6 import android.util.Log;
7 import android.view.MenuItem;
8
9 import com.chris.churchapp.NavUtils;
10 import com.chris.churchapp.R;
11 import com.chris.churchapp.fragments.SermonDetailFragment;
12
13 /**
14  * An activity representing a single Sermon detail screen. This
15  * activity is only used on handset devices. On tablet-size devices,
16  * item details are presented side-by-side with a list of items
17  * in a {@link com.chris.churchapp.activities.SermonListActivity}.
18  * <p>
19  * This activity is mostly just a 'shell' activity containing nothing
20  * more than a {@link com.chris.churchapp.fragments.SermonDetailFragment}.
21  */
22 public class SermonDetailActivity extends Activity {
23
24     private static final String TAG = "SermonDetailActivity";
25
26     Long sermonIndex;
27
28     @Override
29     protected void onCreate(Bundle savedInstanceState) {
30         super.onCreate(savedInstanceState);
31         setContentView(R.layout.activity_sermon_detail);
32         Log.i(TAG, "onCreate");
33
34         sermonIndex = Long.valueOf(1);
35
36         // Show the Up button in the action bar.
37         getActionBar().setDisplayHomeAsUpEnabled(true);
38
39         if(savedInstanceState == null) {
40
41             Bundle args = new Bundle();
42             args.putLong(SermonDetailFragment.ARG_ITEM_ID, getIntent()
43                 .getLongExtra(SermonDetailFragment.ARG_ITEM_ID, 1));
44             //Place a sermonDetailFragment as our Content pane
45             SermonDetailFragment frag = new SermonDetailFragment();
46             frag.setArguments(args);
47             getFragmentManager().beginTransaction().add(R.id.sermon_detail_container, frag).commit();
48
49         }

```

```
50
51 }
52
53 @Override
54 public boolean onOptionsItemSelected(MenuItem item) {
55     switch (item.getItemId()) {
56         case android.R.id.home:
57             // This ID represents the Home or Up button. In the case of this
58             // activity, the Up button is shown. Use NavUtils to allow users
59             // to navigate up one level in the application structure. For
60             // more details, see the Navigation pattern on Android Design:
61             //
62             // http://developer.android.com/design/patterns/navigation.html#up-vs-back
63             //
64             NavUtils.navigateUpTo(this, new Intent(this, SermonListActivity.class));
65             return true;
66         }
67     return super.onOptionsItemSelected(item);
68 }
69 }
70
```

```

1 package com.chris.churchapp.activities;
2
3 import android.app.Activity;
4 import android.app.ListActivity;
5 import android.app.LoaderManager;
6 import android.app.SearchManager;
7 import android.content.CursorLoader;
8 import android.content.Intent;
9 import android.content.Loader;
10 import android.database.Cursor;
11 import android.graphics.Typeface;
12 import android.net.Uri;
13 import android.os.Bundle;
14 import android.provider.SearchRecentSuggestions;
15 import android.text.format.DateUtils;
16 import android.util.Log;
17 import android.view.View;
18 import android.widget.AdapterView;
19 import android.widget.ImageView;
20 import android.widget.ListView;
21 import android.widget.SimpleCursorAdapter;
22 import android.widget.TextView;
23
24 import com.chris.churchapp.R;
25 import com.chris.churchapp.database.SermonSuggestionProvider;
26 import com.chris.churchapp.database.SermonsContract;
27 import com.chris.churchapp.fragments.SermonDetailFragment;
28 import com.nostra13.universalimageloader.core.ImageLoader;
29
30 public class SermonSearchActivity extends Activity implements
31     LoaderManager.LoaderCallbacks<Cursor> {
32
33     private final String TAG = "SermonSearchActivity";
34     private SimpleCursorAdapter adapter;
35
36     private static final String[] FROM = {SermonsContract.Columns.TITLE, SermonsContract.Columns.PUBDATE, .
37     private static final int TO[] = {R.id.sermon_title_textview, R.id.sermon_date_textview, R.id.sermon_thumbnai
38     l_imageview};
39
40     private ListView listView;
41     private TextView textView;
42
43     @Override
44     public void onCreate(Bundle savedInstanceState) {
45         super.onCreate(savedInstanceState);
46         Log.v(TAG, "onCreate()");
47         setContentView(R.layout.sermon_search_activity);
48
49         textView = (TextView) findViewById(R.id.text);

```

```

49     listView = (ListView) findViewById(R.id.list);
50
51     // Create a new adapter and bind it to the List View
52     adapter = new SimpleCursorAdapter(
53         this,
54         R.layout.sermon_list_item,
55         null,
56         FROM,
57         TO,
58         0
59     );
60     adapter.setViewBinder(new SermonViewBinder());
61     listView.setAdapter(adapter);
62
63     // Initiate the Cursor Loader
64     getLoaderManager().initLoader(0, null, this);
65
66     // Get the launch Intent
67     parseIntent(getIntent());
68 }
69
70 /** Handles custom binding of data to view. */
71 class SermonViewBinder implements SimpleCursorAdapter.ViewBinder {
72
73     @Override
74     public boolean setViewValue(View view, Cursor cursor, int columnIndex) {
75         if (view.getId() == R.id.sermon_title_textview) {
76             if (cursor.getInt(
77                 cursor.getColumnIndex(
78                     SermonsContract.Columns.WATCHED_STATE)) == 0) {
79                 ((TextView) view).setTypeface(null, Typeface.BOLD);
80             }
81             else {
82                 ((TextView) view).setTypeface(null, Typeface.NORMAL);
83             }
84         }
85         if (view.getId() == R.id.sermon_date_textview) { // R.id.item_subtitle {
86             // Convert timestamp to relative time
87             Long timestamp = cursor.getLong(columnIndex);
88             CharSequence relativeTime = DateUtils
89                 .getRelativeTimeSpanString(timestamp);
90             ((TextView) view).setText(relativeTime);
91
92             return true;
93         }
94         else if (view.getId() == R.id.sermon_thumbnail_imageview) {
95             ImageLoader.getInstance().displayImage(cursor.getString(columnIndex),
96                 (ImageView) view);
97

```

```

98         return true;
99     } else return false;
100 }
101 }
102
103 @Override
104 protected void onNewIntent(Intent intent) {
105     super.onNewIntent(intent);
106     parseIntent(getIntent());
107 }
108
109 private static String QUERY_EXTRA_KEY = "QUERY_EXTRA_KEY";
110
111 private void parseIntent(Intent intent) {
112     // If the Activity was started to service a Search request.
113     // extract the search query.
114     if (Intent.ACTION_SEARCH.equals(intent.getAction())) {
115         String searchQuery = intent.getStringExtra(SearchManager.QUERY);
116
117         //Setup Recent Suggestions
118         SearchRecentSuggestions suggestions = new SearchRecentSuggestions(
119             this,
120             SermonSuggestionProvider.AUTHORITY,
121             SermonSuggestionProvider.MODE);
122
123         suggestions.saveRecentQuery(searchQuery, null);
124
125         // Perform the search, passing in the search query as an argument
126         // to the Cursor Loader
127         Bundle args = new Bundle();
128         args.putString(QUERY_EXTRA_KEY, searchQuery);
129
130         // Restart the Cursor Loader to execute the new query.
131         getLoaderManager().restartLoader(0, args, this);
132     }
133 }
134
135 public Loader<Cursor> onCreateLoader(int id, Bundle args) {
136     String query = "0";
137
138     if (args != null) {
139         // Extract the search query from the arguments.
140         query = args.getString(QUERY_EXTRA_KEY);
141     }
142
143     // Construct the new query in the form of a Cursor Loader.
144     String[] projection = {SermonsContract.Columns.ID, SermonsContract.Columns.TITLE};
145     String where = SermonsContract.Columns.TITLE
146         + " LIKE \"%\" + query + \"%\"";

```

```

147 String[] whereArgs = null;
148 // String sortOrder = SermonsContract + " COLLATE LOCALIZED ASC";
149
150 // // Define the on-click listener for the list items
151 listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
152
153     @Override
154     public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
155         // Build the Intent used to open WordActivity with a specific word Uri
156         Intent sermonsIntent = new Intent(getApplicationContext(), SermonDetailActivity.class);
157         // Uri data = Uri.withAppendedPath(SermonsContract.Columns.CONTENT_URI, String.valueOf(id));
158         sermonsIntent.putExtra(SermonDetailFragment.ARG_ITEM_ID, id);
159         startActivity(sermonsIntent);
160     }
161 });
162
163 // Create the new Cursor loader.
164 return new CursorLoader(this, SermonsContract.Columns.CONTENT_URI,
165     null, where, whereArgs,
166     SermonsContract.SORT_ORDER);
167 }
168
169 public void onLoadFinished(Loader<Cursor> loader, Cursor cursor) {
170     // Replace the result Cursor displayed by the Cursor Adapter with
171     // the new result set.
172     Log.v(TAG, "OnLoadFinished: Count=" + cursor.getCount());
173     if(cursor.getCount() == 0){
174         textView.setText("No results found");
175     }
176     else {
177         textView.setText(cursor.getCount() + " results found");
178     }
179     adapter.swapCursor(cursor);
180 }
181
182 public void onLoaderReset(Loader<Cursor> loader) {
183     // Remove the existing result Cursor from the List Adapter.
184     adapter.swapCursor(null);
185 }
186
187 }

```